FuseTune: Low-Budget Hyperparameter Tuning for Fused Regression

First Author[1*†] and Second Author[1]

[1*]Department, Institution, Street Address, City, 00000, State, Country.

*Corresponding author(s). E-mail(s): first.author@example.com;
Contributing authors: second.author@example.com;
†These authors contributed equally to this work.

Abstract

Fused regression models rely on two coupled hyperparameters: a sparsity penalty and a fusion penalty. In practice these hyperparameters are often tuned by exhaustive grids or warm-start heuristics, even though each cross-validation evaluation can be expensive and the validation surface can be noisy, solver-dependent, and strongly dataset-dependent. We present FuseTune, a low-budget tuning procedure for fused regression that searches in log-parameter space and adapts its strategy to the solver. For the $\boldsymbol{\ell_1}$ fused solver, FuseTune uses a nonmonotone axis-aligned direct search. For the $\boldsymbol{\ell_2}$ fused solver, it uses a redesigned `glmnet`-seeded hybrid search that screens a small set of candidates before conditional local refinement. Across nine real $\boldsymbol{\ell_1}$ datasets, FuseTune is faster than `glmnet`-seeded on all datasets while matching or improving test RMSE on seven. On nine real $\boldsymbol{\ell_2}$ datasets, the redesigned FuseTune is faster than `glmnet`-seeded on eight datasets and nearly ties its mean test RMSE while using less than half the average number of validation evaluations. Synthetic regime studies show that FuseTune is the strongest efficiency method for $\boldsymbol{\ell_1}$, and a credible low-budget alternative for $\boldsymbol{\ell_2}$, especially in regimes where fusion is helpful. The contribution is practical rather than theoretical: we do not claim a new optimization class, but a reproducible, solver-aware tuning strategy for fused regression.

Keywords: fused regression, hyperparameter tuning, derivative-free optimization, cross-validation, regularization

1 Introduction

Fused regression is attractive when related groups may share coefficient structure without being identical. A fused model allows each group to retain its own coefficients while borrowing strength through a penalty on inter-group differences. This idea is useful when partial similarity is plausible but complete pooling is too strong. The practical difficulty is that fused regression introduces at least two interacting hyperparameters: a sparsity penalty λ and a fusion penalty γ. Their joint choice is usually made by cross-validation, yet cross-validation is exactly where the computational bottleneck appears.

The tuning problem is awkward for three reasons. First, each objective evaluation is expensive because it nests model fitting inside repeated train-validation splits. Second, the cross-validation surface can be non-smooth or effectively noisy because folds, group structure, and solver

failures all affect the observed score. Third, the preferred region of (λ, γ) is not stable across datasets or even across solver families. In some datasets fusion is clearly helpful; in others the best choice is close to no fusion at all. A useful tuner therefore needs to be cheap, robust, and cautious about spending evaluation budget where the surface gives little return.

This paper studies that tuning problem empirically and proposes FuseTune, a solver-aware low-budget tuner for fused regression. The method is deliberately modest. For the ℓ_1 fused solver it performs a nonmonotone axis-aligned direct search in log-parameter space. For the ℓ_2 fused solver it uses a different design: a `glmnet`-seeded hybrid search that screens a few candidates, refines γ first, and only touches λ locally when the screening stage justifies it. The central claim is not that FuseTune defines a new optimization class. The claim is that fused-regression tuning benefits from a small, solver-aware search procedure that spends the validation budget where it matters.

Our main empirical findings are straightforward. On the retained real-data ℓ_1 suite, FuseTune is faster than `glmnet`-seeded on all nine datasets and no worse in test RMSE on seven. On the retained real-data ℓ_2 suite, the redesigned FuseTune is faster than `glmnet`-seeded on eight of nine datasets, has lower mean test RMSE by a negligible margin, and uses roughly 9.6 validation evaluations on average versus 20.4 for `glmnet`-seeded. On synthetic regimes, FuseTune is the strongest efficiency method for ℓ_1, and after the ℓ_2 redesign it becomes a credible low-budget alternative there as well, although not the raw-RMSE winner on average.

In summary, the contributions of this paper are:

- A solver-aware, low-budget tuning procedure for fused regression that adapts its outer search to the inner solver: nonmonotone axis-aligned direct search for the ℓ_1 solver and a `glmnet`-seeded hybrid for the ℓ_2 solver.
- A normalized gain-per-second efficiency metric that makes the accuracy–budget trade-off explicit across datasets and methods.
- An empirical benchmark on nine real datasets per solver and 30 synthetic regimes showing that FuseTune is consistently faster than the warm-start baseline while retaining nearly all predictive performance.

The paper is organized as follows. Section 2 positions the work relative to fused penalties and derivative-free search. Section 3 presents the problem setup, FuseTune, and the efficiency metric used in the comparisons. Section 4 summarizes the experimental design. Section 5 presents the real-data, synthetic, and surface-diagnostic results. Section 6 closes with practical interpretation and limitations.

2 Related Work

Classical regularized regression starts with the lasso and extends naturally to nonconvex, grouped, and mixed penalties [1–6]. These papers matter here for two reasons. First, they establish the statistical motivation for imposing structured shrinkage rather than treating each coefficient independently. Second, they show that once a model carries multiple regularization terms, the outer hyperparameter search becomes a central practical problem rather than a minor implementation detail.

Fusion penalties and related coefficient-clustering formulations form the immediate statistical background. The fused lasso itself couples sparsity and local smoothness [7], while later work developed pathwise coordinate methods, specialized path algorithms, generalized-lasso formulations, and efficient implementations for broader fused problems [8–12]. Closely related ideas appear in predictor clustering and categorical fusion penalties [13, 14], multivariate shared-support estimation [15], and multi-class graphical models with fused penalties [16, 17]. Our paper does not propose another fused estimator. It studies the outer tuning problem that remains once such an estimator is fixed.

That outer problem sits squarely in the literature on model selection and hyperparameter optimization. Cross-validation is classical [18] and widely surveyed [19], but repeated model comparison over the same validation protocol can induce instability and optimistic bias [20–22]. Modern hyperparameter optimization offers generic alternatives, including random search [23], tree-structured Parzen estimators [24], Gaussian-process Bayesian optimization and efficient global

optimization [25, 26], sequential model-based configuration [27], budgeted methods such as Hyperband [28], and software frameworks such as Optuna [29]; see also the broader review by Feurer and Hutter [30]. These methods are useful reference points, but they are generic. They do not exploit the solver-specific structure of fused regression, and they can still spend substantial budget when each objective call requires repeated grouped fits.

Derivative-free optimization provides a more relevant algorithmic lens. Classical direct-search and simplex methods include Hooke–Jeeves [31], Nelder–Mead [32], pattern-search variants [33, 34], and mesh adaptive direct search [35]. Broader treatments appear in the reviews and monographs of Kolda et al., Conn et al., Audet and Hare, and Kelley [36–39]. By contrast, bound-constrained quasi-Newton methods such as L-BFGS-B [40, 41] assume smoother local behavior. That distinction is exactly what matters in our setting: if the cross-validation surface behaves smoothly enough, gradient information should help; if it is noisy or effectively nonsmooth, a careful low-budget derivative-free policy should be more reliable.

Within that landscape, FuseTune is best viewed as a practical search policy built from familiar ingredients rather than a new optimizer class. The main contribution is empirical and procedural: a solver-aware low-budget tuner for fused regression, together with a benchmark that makes the trade-off between predictive performance and validation budget explicit.

3 Problem Setup and FuseTune

3.1 Fused regression tuning problem

Suppose the data are partitioned into K groups, with design matrix $X_g \in \mathbb{R}^{n_g \times p}$, response $y_g \in \mathbb{R}^{n_g}$, and coefficient vector $\beta_g \in \mathbb{R}^p$ for group g. A generic fused regression problem can be written as

$$\begin{aligned} \min_{\{\beta_g\}_{g=1}^K} \quad & \sum_{g=1}^{K} \frac{1}{2n_g} \|y_g - X_g\beta_g\|_2^2 \\ & + \lambda \sum_{g=1}^{K} \|\beta_g\|_1 \\ & + \gamma \sum_{(u,v)\in E} w_{uv}\, \phi(\beta_u - \beta_v), \end{aligned} \tag{1}$$

where E is the set of fused group pairs, w_{uv} are fusion weights, and ϕ determines the fusion geometry. In the ℓ_1 fused solver implemented in this repository, $\phi(d) = \|d\|_1$, which matches the objective evaluated in the helper routine `.objective_l1_fused`. In the ℓ_2 solver, fusion is encoded by appending difference rows to the design matrix and solving an augmented sparse regression problem with `glmnet`; the resulting fusion term is quadratic in the coefficient differences, so $\phi(d)$ is proportional to $\|d\|_2^2$. Equation (1) is therefore a compact summary of what both solvers optimize, while leaving the exact fusion geometry solver-specific.

The tuning problem is to choose (λ, γ) by minimizing the mean validation RMSE over fixed folds. Because both hyperparameters are strictly positive in the implementation, FuseTune searches in log-parameter space:

$$\theta = (\log \lambda, \log \gamma). \tag{2}$$

This reparameterization respects positivity constraints and makes multiplicative moves in (λ, γ) correspond to additive moves in θ.

3.2 The FuseTune search policy

The ℓ_1 and ℓ_2 solvers in this repository behave differently enough that one outer tuner was not ideal for both. The current implementation therefore uses two branches under a shared interface. The ℓ_1 branch starts near the center of the search box in log-space, probes coordinate directions opportunistically, accepts nonmonotone improvements relative to a short rolling history, and adapts the step size by shrinkage and mild expansion. The ℓ_2 branch instead begins from a `glmnet`-derived λ seed, screens a small set of nearby λ values against a short γ grid, refines γ first, and only performs a

local λ touch-up when the screening step justifies it.

Figure 1 illustrates both search trajectories on a stylized validation surface. Both branches optimize the same cached cross-validation objective, but they navigate the $(\log\lambda, \log\gamma)$ space differently: the ℓ_1 branch explores via axis-aligned coordinate moves from the box center, while the ℓ_2 branch anchors at a `glmnet`-derived λ seed and spends most of its budget refining γ.

Algorithm 1 summarizes the resulting policy.

3.3 Efficiency metric

Raw test RMSE, elapsed time, and validation-evaluation count are all useful, but none alone captures the intended trade-off. We therefore summarize many comparisons with a normalized gain-per-second score. For method m on dataset or regime d, let $r_{m,d}$ denote test RMSE, let $r_{\text{feat},d}$ be the featureless baseline RMSE, and let $r_{\star,d}$ be the best RMSE achieved by any compared method on that dataset or regime. The normalized gain is

$$G_{m,d} = \max\left(0, \min\left(1, \frac{r_{\text{feat},d} - r_{m,d}}{r_{\text{feat},d} - r_{\star,d}}\right)\right), \quad (3)$$

and the efficiency summary is

$$\text{GPS}_{m,d} = \frac{G_{m,d}}{t_{m,d}}, \quad (4)$$

where $t_{m,d}$ is elapsed time in seconds. Higher values are better. If no method improves over the featureless baseline on dataset d, we set $G_{m,d} = 0$ for all methods on that dataset. This score avoids rewarding the featureless model for being cheap while still penalizing methods that achieve little predictive gain.

4 Experimental Design

Throughout this section, "retained" refers to the final set of methods, datasets, and regimes kept for the main-text comparisons after preliminary screening excluded methods that were either redundant or consistently dominated. The supplementary baselines excluded from the retained panel are reported only in the appendix.

Algorithm 1 FuseTune

Require: training data split into groups, solver type, bounds for λ and γ, cross-validation objective $f(\log\lambda, \log\gamma)$, evaluation budget

if solver is ℓ_1 **then**
  initialize $x = (\log\lambda, \log\gamma)$ at the box center
  initialize axis step sizes and a short nonmonotone history buffer
  while budget remains and step sizes exceed threshold **do**
    evaluate coordinate moves around x in a preferred order
    if a candidate improves relative to the nonmonotone reference **then**
      accept the candidate, update the history buffer, and adapt the successful axis step
    else
      shrink the step sizes
    end if
    if progress stays flat for several iterations **then**
      terminate
    end if
  end while
else
  estimate a `glmnet`-based seed for $\log\lambda$
  screen a small set of nearby $\log\lambda$ values on a short $\log\gamma$ grid
  retain the best screened point and refine $\log\gamma$ locally
  if the screened point moved meaningfully away from the seed or the γ refinement improves the score **then**
    test a small local neighborhood in $\log\lambda$
  end if
end if
return the best $(\log\lambda, \log\gamma)$ found

4.1 Method panel

The main paper uses a fixed retained panel: `featureless`, `grid`, `random`, `hooke_jeeves`, `lbfgsb_multistart`, `glmnet_seeded`, and `fusetune`. The `featureless` model provides a null reference. Grid and random search represent exhaustive and stochastic black-box search, with random search motivated by the broader hyperparameter-optimization literature [23].

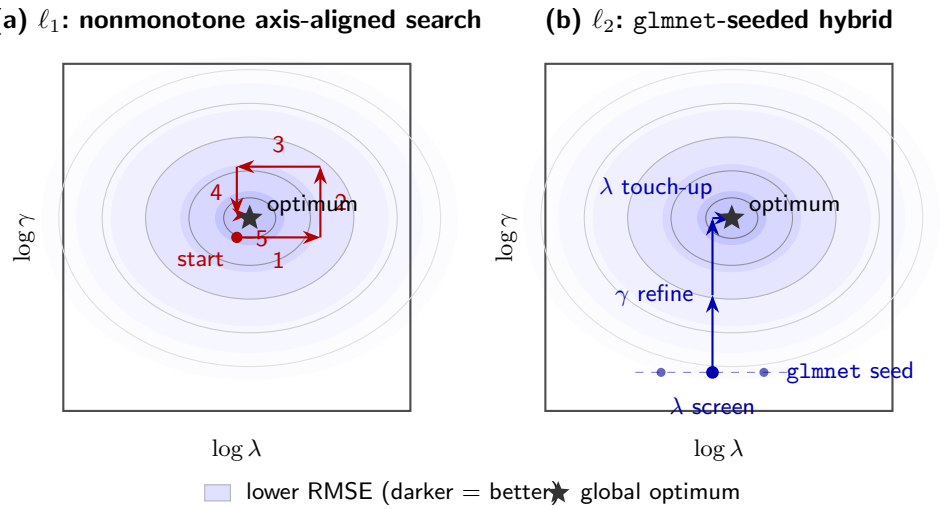


Fig. 1: Search trajectories of FUSETUNE on a stylized validation surface in $(\log \lambda, \log \gamma)$ space. Contour lines represent cross-validation RMSE levels; darker shading indicates lower (better) values. (a) The ℓ_1 branch starts at the box center and navigates via axis-aligned coordinate moves with nonmonotone acceptance. (b) The ℓ_2 branch starts from a `glmnet`-derived λ seed, screens a small set of λ candidates at low γ, refines γ vertically, and adjusts λ only when justified.

Hooke–Jeeves and L-BFGS-B represent classical derivative-free and gradient-based baselines [31, 40, 41]. `glmnet`-seeded search is the strongest structure-exploiting baseline for the current code base because the ℓ_2 implementation is itself built around a `glmnet` path [42]. Extra synthetic baselines such as `bayesian_opt` are reported only in the appendix, where they are interpreted as generic global-search comparators rather than direct methodological rivals [25, 26].

4.2 Real-data suites

The retained ℓ_1 suite contains nine real datasets and compares `fusetune` against `glmnet_seeded`. The retained ℓ_2 suite uses the same nine datasets with the full retained method panel. We report test RMSE, elapsed time, and validation-evaluation count from the aggregated summaries in the repository. For the ℓ_1 suite, the key operational comparison is `fusetune` versus `glmnet_seeded`; for the ℓ_2 suite, we report both that head-to-head and the broader baseline context.

4.3 Synthetic regime benchmark

The synthetic benchmark contains 30 realized regimes across three families: `fusion_helpful_chain`, `heavy_tailed_outlier`, and `sparse_shared_noisy`. The benchmark is valuable because it separates regimes where fusion should be useful from regimes where the search problem is more adversarial. For ℓ_2, all 30 regimes completed under the updated benchmark after rerunning the two hardest sparse-noisy cases with a higher timeout.

4.4 Surface diagnostics

To make the search problem tangible, we include two ℓ_2 validation-surface diagnostics. The `school_ilea` surface is useful because it prefers very small fusion. The `beijing_air_quality` surface is useful because it prefers substantial fusion and an interior optimum once the search range is widened. Together they show that γ is neither universally negligible nor universally dominant in the same direction.

5 Results

5.1 Method roles

Table 1 summarizes the retained comparison panel and the budget rationale.

Table 1: Retained method panel and roles.

Method	Role in the comparison
`featureless`	Null baseline with no tuning cost.
`grid`	Exhaustive reference over the search box.
`random`	Stochastic black-box baseline.
`hooke_jeeves`	Classical derivative-free local search.
`lbfgsb_multistart`	Gradient-based bound-constrained baseline.
`glmnet_seeded`	Structure-exploiting warm-start baseline.
`fusetune`	Proposed low-budget solver-aware tuner.

5.2 Real-data ℓ_1 results

The ℓ_1 results are the cleanest part of the paper. Across the nine retained real datasets, `fusetune` is faster than `glmnet_seeded` on all nine datasets, uses less memory on seven, and achieves test RMSE no worse than the baseline on seven. Five datasets are triple-dominance cases in which `fusetune` is faster, lighter, and at least as accurate. The mean elapsed time drops from 6.53 seconds for `glmnet_seeded` to 3.25 seconds for `fusetune`.

Table 2 shows the dataset-level pattern. The RMSE differences are usually tiny, but the time savings are systematic. The strongest wins appear on `rdatasets_dietox`, `world_bank_wdi`, and `beijing_air_quality`. The only clear counterexample on accuracy is `rdatasets_grunfeld`, where `glmnet_seeded` attains a slightly lower test RMSE.

5.3 Real-data ℓ_2 results

The ℓ_2 story is more nuanced. The redesigned `fusetune` is not the raw-RMSE winner on average across the full method panel, but it is now a credible efficiency method. Relative to `glmnet_seeded`, it is faster on eight of nine datasets, lower in memory on seven, and no worse in RMSE on six. Mean elapsed time drops from 1.585 seconds to 1.059 seconds, while mean test RMSE is essentially tied at 18389.207 for `glmnet_seeded` and 18389.201 for `fusetune`.

The strongest ℓ_2 gains for `fusetune` occur on `communities_crime`, `king_county_house_sales`, and `rdatasets_dietox`. The main caveat is that raw gain-per-second can still favor `lbfgsb_multistart` because it is extremely cheap; however, that method achieves RMSE no worse than `glmnet_seeded` on only four of nine datasets, which makes it a less reliable practical choice. Figure 2 summarizes the head-to-head comparison between `fusetune` and `glmnet_seeded`.

5.4 Synthetic results

The synthetic benchmark clarifies when `fusetune` helps and when it mainly trades accuracy for budget. For ℓ_1, `fusetune` is the top efficiency method by normalized gain per second and is especially strong in the `fusion_helpful_chain` family, where it achieves the best mean RMSE among the retained methods and the lowest time among the serious competitors. In `heavy_tailed_outlier`, `lbfgsb_multistart` attains the lowest mean RMSE, but `fusetune` remains comparably cheap. In `sparse_shared_noisy`, `glmnet_seeded` has a small RMSE edge, whereas `fusetune` retains a clear time advantage.

For ℓ_2, the redesigned `fusetune` changes the story materially. The earlier implementation oversearched the validation surface and was too expensive to justify. After replacing the coarse two-dimensional search with the current `glmnet`-seeded hybrid, mean gain per second rises to 2.52, the highest among the synthetic ℓ_2 methods. That does not make `fusetune` the accuracy winner: `hooke_jeeves` still has the lowest mean RMSE across the completed benchmark, and `grid` remains strong in difficult sparse-noisy regimes. The redesign instead turns `fusetune` into a credible low-budget method.

Table 4 highlights the family-level picture, and Figure 3 summarizes the gain-per-second ranking across families.

5.5 Surface diagnostics

The surface diagnostics explain why a single search policy was not ideal for both solvers. Figure 4 shows train and validation marginals for `school_ilea`, ℓ_2 and `beijing_air_quality`, ℓ_2. In `school_ilea`, the preferred γ region is near the lower edge, which is consistent with the broader observation that fusion can be neutral or slightly

Table 2: Real-data ℓ_1 comparison between `glmnet_seeded` and `fusetune`. Speedup is relative to `glmnet_seeded`.

Dataset	RMSE (`glmnet`)	RMSE (`fusetune`)	Time (`glmnet`)	Time (`fusetune`)	Speedup
Beijing Air Quality	52.255	52.184	2.867	0.889	3.22
Communities and Crime	0.192	0.191	23.663	14.898	1.59
King County House Sales	389453.206	389453.206	8.957	3.071	2.92
OWID CO_2	8.043	8.043	0.220	0.062	3.55
Radon Minnesota	0.791	0.791	6.132	4.020	1.53
RDatasets Dietox	13.853	13.845	7.600	1.522	4.99
RDatasets Grunfeld	52.886	53.029	1.884	0.652	2.89
School ILEA	6.325	6.325	7.227	4.089	1.77
World Bank WDI	9682.956	9682.956	0.250	0.061	4.10

Table 3: Real-data ℓ_2 results. The best RMSE method is reported over the retained method panel.

Dataset	Best method	Best RMSE	RMSE (`glmnet`)	RMSE (`fusetune`)	Time (`glmnet`)	Time (`fusetune`)
Beijing Air Quality	`grid`	34.3362	34.3375	34.3377	0.337	0.210
Communities and Crime	`grid`	0.1320	0.1398	0.1374	10.551	7.500
King County House Sales	`lbfgsb_multistart`	164497.954	164498.531	164498.531	1.964	0.694
OWID CO_2	`hooke_jeeves`	0.5392	0.5416	0.5401	0.238	0.157
Radon Minnesota	`hooke_jeeves`	0.7210	0.7519	0.7433	0.222	0.187
RDatasets Dietox	`fusetune`	3.4793	3.5210	3.4793	0.206	0.272
RDatasets Grunfeld	`lbfgsb_multistart`	48.9341	48.9364	48.9364	0.172	0.146
School ILEA	`glmnet_seeded`	6.2878	6.2878	6.2890	0.380	0.231
World Bank WDI	`hooke_jeeves`	909.720	909.818	909.818	0.196	0.132

harmful there. In `beijing_air_quality`, the preferred γ region is clearly interior and substantially away from zero once the search range is widened. The two plots therefore support the same practical conclusion: the validation landscape is highly dataset-dependent, and the tuner needs to be prepared both for near-zero fusion and for clearly nontrivial fusion.

6 Discussion

Three practical conclusions follow from the experiments. First, ℓ_1 tuning is where `fusetune` is strongest. The method is consistently faster than the warm-start baseline and usually indistinguishable from it in test RMSE. Second, ℓ_2 tuning benefits from a different outer-search policy. The original ℓ_2 branch of `fusetune` spent too much budget on coarse two-dimensional exploration; the redesigned branch works better precisely because it borrows a `glmnet` seed and spends most of its budget screening γ quickly. Third, the validation landscape is not universally smooth or universally hostile. It changes with the solver and with the data regime. That is why a single generic tuning rule was a poor fit in practice.

The role of `lbfgsb_multistart` deserves separate comment. On both real and synthetic ℓ_2 benchmarks it can appear very efficient because it is cheap, and on real ℓ_2 it attains the highest raw mean gain-per-second score. Yet its accuracy is not reliably competitive: in the real ℓ_2 retained suite it is no worse than `glmnet_seeded` on only four of nine datasets. We therefore view it as a useful stress-test baseline rather than the preferred practical choice.

In that sense, FuseTune is a conservative contribution. It does not attempt to replace established optimization theory. It offers an implementation-level answer to a recurring empirical problem: when fused-regression tuning is expensive, a small solver-aware search policy can outperform exhaustive tuning in wall-clock cost while retaining nearly all of the predictive performance.

7 Conclusion

We have presented FuseTune, a solver-aware low-budget hyperparameter tuner for fused regression. The method is deliberately modest: it does not introduce new optimization theory, but rather shows that a small, solver-specific search policy can systematically reduce tuning cost while retaining nearly all of the predictive performance of more expensive alternatives. On the ℓ_1 side, the gains are clear and consistent; on the ℓ_2

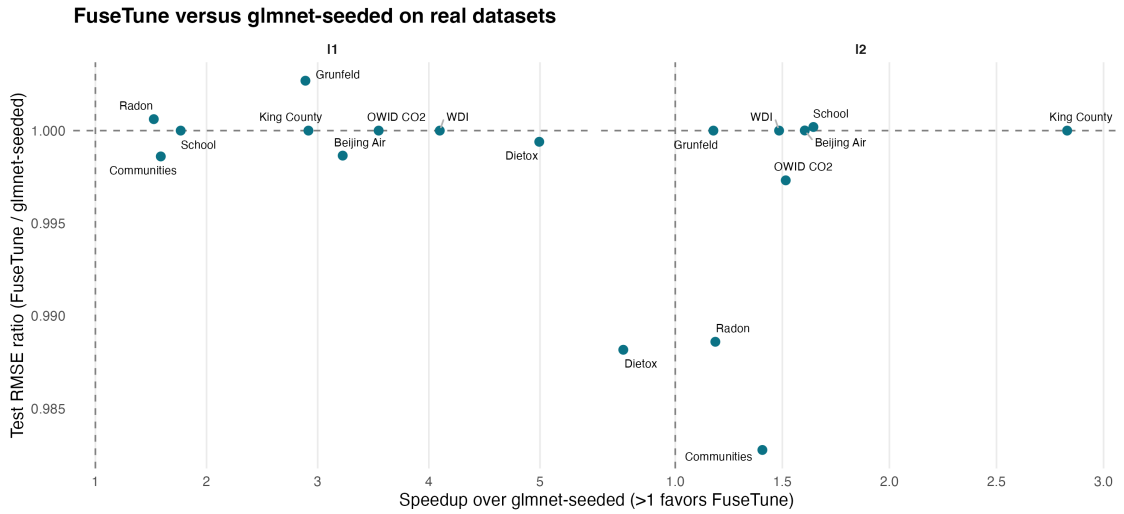


Fig. 2: Real-data comparison between `fusetune` and `glmnet_seeded`. Each point is a dataset. Values to the right of the vertical line indicate that `fusetune` is faster; values below the horizontal line indicate that `fusetune` attains lower test RMSE. The ℓ_1 panel shows a clear practical win for `fusetune`. The ℓ_2 panel is more mixed on RMSE, but still strongly favors `fusetune` on elapsed time.

Table 4: Synthetic family summary using retained baselines only. The "best retained baseline" column excludes `bayesian_opt`, which is reported in the appendix.

Solver	Family	Best retained baseline	Baseline RMSE	Baseline time	`fusetune` RMSE	`fusetune` time / evals
ℓ_1	Fusion-helpful chain	`glmnet_seeded`	0.7059	1.907	0.7055	0.836 / 8.1
ℓ_1	Heavy-tailed outlier	`lbfgsb_multistart`	1.8031	9.116	1.8216	8.926 / 9.5
ℓ_1	Sparse shared noisy	`glmnet_seeded`	1.2707	36.641	1.2784	22.202 / 14.2
ℓ_2	Fusion-helpful chain	`hooke_jeeves`	0.6072	0.805	0.6061	0.323 / 10.1
ℓ_2	Heavy-tailed outlier	`hooke_jeeves`	1.2052	4.104	1.2297	1.925 / 9.2
ℓ_2	Sparse shared noisy	`hooke_jeeves`	1.1280	11.097	1.2005	2.929 / 8.8

side, the redesigned hybrid branch offers a credible efficiency–accuracy compromise. We hope the accompanying benchmark and efficiency metric encourage further work on practical tuning strategies for structured regularization problems.

8 Limitations

The paper has four important limitations. First, the evidence is empirical and does not establish convergence guarantees for `fusetune`. Second, the retained real-data suites are large enough to be informative, but still limited to nine datasets per solver. Third, the ℓ_2 results are stronger on efficiency than on raw accuracy, so the method should not be marketed as the universally best ℓ_2 tuner. Fourth, the exact solver behavior is tied to this code base, including the current `glmnet`-based ℓ_2 implementation and the repository's fused ℓ_1 solvers. A different fused-regression implementation could lead to different tuning surfaces and therefore different outer-search behavior.

Acknowledgements. Not applicable.

Declarations

Funding Not applicable.

Conflict of interest/Competing interests The authors declare no competing interests.

Ethics approval and consent to participate Not applicable.

Consent for publication Not applicable.

Data availability The real-data benchmark inputs are stored as processed grouped-regression files in the accompanying repository at https://github.com/EngineerDanny/fuserplus. The synthetic benchmark inputs are generated from the regime manifests included in the repository.

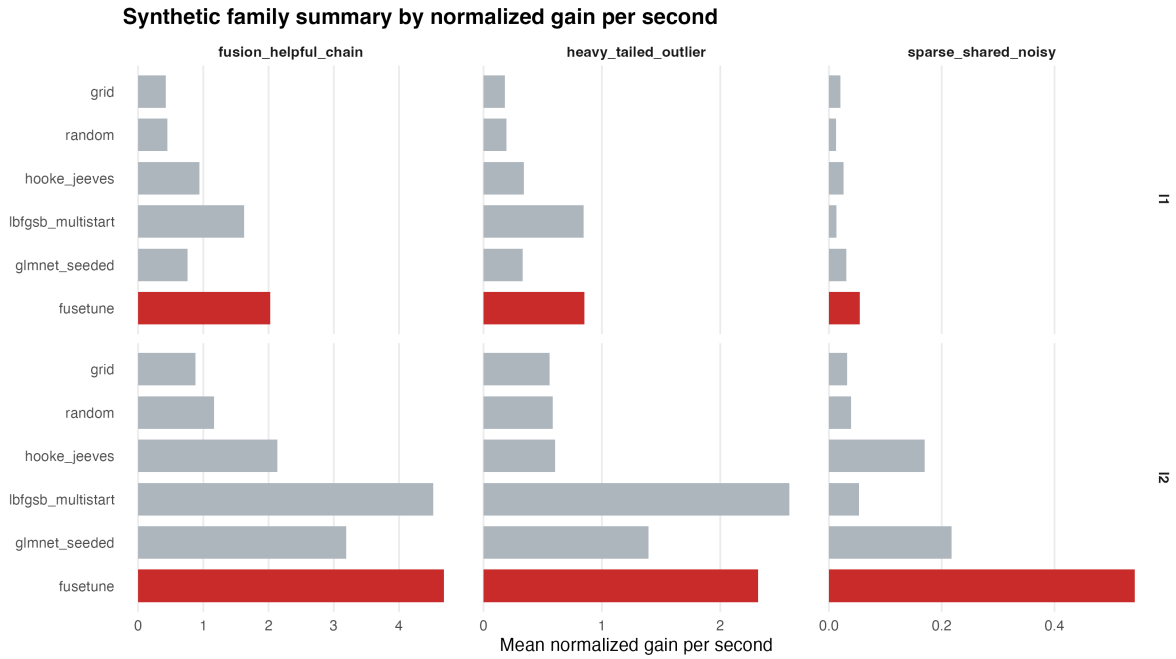
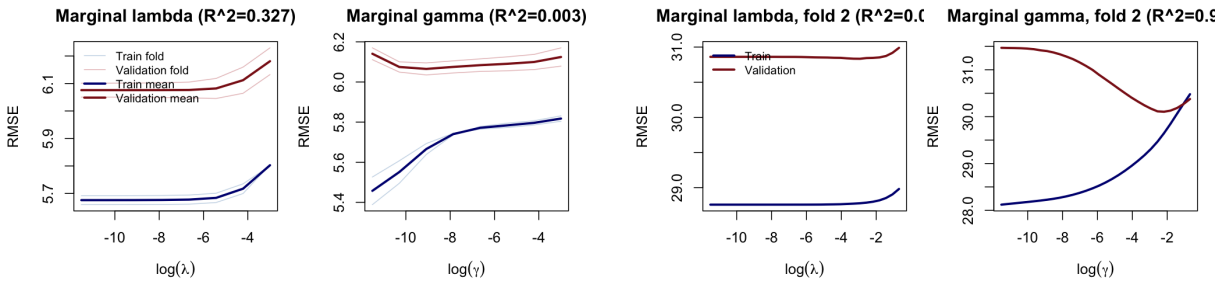


Fig. 3: Synthetic family comparison by normalized gain per second. The ℓ_1 panels show a clear efficiency advantage for `fusetune`. The redesigned ℓ_2 panels show the same pattern in `fusion_helpful_chain` and a strong cost-quality compromise in the other two families.



(a) `school_ilea`, ℓ_2 (b) `beijing_air_quality`, ℓ_2

Fig. 4: ℓ_2 validation-surface diagnostics for two representative datasets. (a) shows per-fold marginals together with their across-fold means; (b) shows single-fold marginals. In both cases the preferred regularization regime is not stable across datasets: `school_ilea` pushes the optimum toward very weak fusion, while `beijing_air_quality` favors a clearly positive fusion level.

Materials availability Not applicable.

Code availability All code used to fit the models, run the tuning benchmarks, and generate the manuscript figures is contained in the accompanying repository. The primary tuning implementation is in `local_tune_lambda_gamma_experiment_round3.R`, and the manuscript figures are generated by `make_paper_figures.R`.

Author contribution Placeholder text. Author names and contributions will be finalized before submission.

Appendix A Supplementary synthetic baselines

Table A1 reports the overall synthetic benchmark summaries including `bayesian_opt`. These numbers are omitted from the main text to keep the main comparison aligned with the retained real-data method panel.

Appendix B Why the ℓ_2 redesign mattered

The original ℓ_2 branch of `fusetune` used a coarse two-dimensional search followed by local refinement. On the synthetic benchmark that design used more than 50 validation evaluations on average and spent enough time exploring weak directions that its low-budget claim was hard to defend. The current branch instead uses a `glmnet`-seeded λ anchor, a small γ screen, and a local λ adjustment only when screening indicates that the seed is not already adequate. On the completed synthetic ℓ_2 benchmark, this redesign reduces the average validation count to 9.37 while preserving competitive predictive performance. The main-text ℓ_2 results should be interpreted as evidence for that redesign, not as evidence that every direct-search variant would behave similarly.

References

[1] Tibshirani, R.: Regression shrinkage and selection via the lasso. Journal of the Royal Statistical Society Series B: Statistical Methodology **58**(1), 267–288 (1996) https://doi.org/10.1111/j.2517-6161.1996.tb02080.x

[2] Fan, J., Li, R.: Variable selection via nonconcave penalized likelihood and its oracle properties. Journal of the American Statistical Association **96**(456), 1348–1360 (2001) https://doi.org/10.1198/016214501753382273

[3] Zou, H., Hastie, T.: Regularization and variable selection via the elastic net. Journal of the Royal Statistical Society Series B: Statistical Methodology **67**(2), 301–320 (2005) https://doi.org/10.1111/j.1467-9868.2005.00503.x

[4] Yuan, M., Lin, Y.: Model selection and estimation in regression with grouped variables. Journal of the Royal Statistical Society Series B: Statistical Methodology **68**(1), 49–67 (2006) https://doi.org/10.1111/j.1467-9868.2005.00532.x

[5] Zou, H.: The adaptive lasso and its oracle properties. Journal of the American Statistical Association **101**(476), 1418–1429 (2006) https://doi.org/10.1198/016214506000000735

[6] Simon, N., Friedman, J., Hastie, T., Tibshirani, R.: A sparse-group lasso. Journal of Computational and Graphical Statistics **22**(2), 231–245 (2013) https://doi.org/10.1080/10618600.2012.681250

[7] Tibshirani, R., Saunders, M., Rosset, S., Zhu, J., Knight, K.: Sparsity and smoothness via the fused lasso. Journal of the Royal Statistical Society Series B: Statistical Methodology **67**(1), 91–108 (2005) https://doi.org/10.1111/j.1467-9868.2005.00490.x

[8] Friedman, J., Hastie, T., Höfling, H., Tibshirani, R.: Pathwise coordinate optimization. The Annals of Applied Statistics **1**(2), 302–332 (2007) https://doi.org/10.1214/07-AOAS131

[9] Hoefling, H.: A path algorithm for the fused lasso signal approximator. Journal of Computational and Graphical Statistics **19**(4), 984–1006 (2010) https://doi.org/10.1198/jcgs.2010.09208

[10] Tibshirani, R.J., Taylor, J.: The solution path of the generalized lasso. The Annals of Statistics **39**(3), 1335–1371 (2011) https://doi.org/10.1214/11-AOS878

[11] Arnold, T.B., Tibshirani, R.J.: Efficient implementations of the generalized lasso dual path algorithm. Journal of Computational and Graphical Statistics **25**(1), 1–27 (2016) https://doi.org/10.1080/

Table A1: Overall synthetic benchmark summaries including `bayesian_opt`.

Solver	Method	Time (s)	Test RMSE	CV evals
ℓ_1	`featureless`	0.004	4.236	0.0
ℓ_1	`lbfgsb_multistart`	8.955	1.350	9.2
ℓ_1	`fusetune`	10.655	1.269	10.6
ℓ_1	`glmnet_seeded`	19.971	1.270	20.7
ℓ_1	`hooke_jeeves`	20.356	1.271	20.8
ℓ_1	`bayesian_opt`	20.483	1.265	20.6
ℓ_1	`random`	34.766	1.291	36.0
ℓ_1	`grid`	34.940	1.268	36.0
ℓ_2	`featureless`	0.005	4.236	0.0
ℓ_2	`fusetune`	1.725	1.012	9.4
ℓ_2	`glmnet_seeded`	2.142	1.058	20.1
ℓ_2	`lbfgsb_multistart`	3.707	1.052	8.8
ℓ_2	`hooke_jeeves`	5.335	0.980	30.7
ℓ_2	`bayesian_opt`	11.873	0.997	28.2
ℓ_2	`grid`	17.402	0.986	36.0
ℓ_2	`random`	18.289	0.986	36.0

10618600.2015.1008638

[12] Liu, J., Yuan, L., Ye, J.: An efficient algorithm for a class of fused lasso problems. In: Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 323–332 (2010). https://doi.org/10.1145/1835804.1835847

[13] Bondell, H.D., Reich, B.J.: Simultaneous regression shrinkage, variable selection, and supervised clustering of predictors with OSCAR. Biometrics **64**(1), 115–123 (2008) https://doi.org/10.1111/j.1541-0420.2007.00843.x

[14] Gertheiss, J., Tutz, G.: Sparse modeling of categorial explanatory variables. The Annals of Applied Statistics **4**(4), 2150–2180 (2010) https://doi.org/10.1214/10-AOAS355

[15] Obozinski, G., Wainwright, M.J., Jordan, M.I.: Support union recovery in high-dimensional multivariate regression. The Annals of Statistics **39**(1), 1–47 (2011) https://doi.org/10.1214/09-AOS776

[16] Friedman, J., Hastie, T., Tibshirani, R.: Sparse inverse covariance estimation with the graphical lasso. Biostatistics **9**(3), 432–441 (2008) https://doi.org/10.1093/biostatistics/kxm045

[17] Danaher, P., Wang, P., Witten, D.M.: The joint graphical lasso for inverse covariance estimation across multiple classes. Journal of the Royal Statistical Society Series B: Statistical Methodology **76**(2), 373–397 (2014) https://doi.org/10.1111/rssb.12033

[18] Stone, M.: Cross-validatory choice and assessment of statistical predictions. Journal of the Royal Statistical Society Series B: Statistical Methodology **36**(2), 111–147 (1974) https://doi.org/10.1111/j.2517-6161.1974.tb00994.x

[19] Arlot, S., Celisse, A.: A survey of cross-validation procedures for model selection. Statistics Surveys **4**, 40–79 (2010) https://doi.org/10.1214/09-SS054

[20] Cawley, G.C., Talbot, N.L.C.: On overfitting in model selection and subsequent selection bias in performance evaluation. Journal of Machine Learning Research **11**, 2079–2107 (2010) https://doi.org/10.5555/1756006.1859921

[21] Varma, S., Simon, R.: Bias in error estimation when using cross-validation for model selection. BMC Bioinformatics **7**(1), 91 (2006) https://doi.org/10.1186/1471-2105-7-91

[22] Ambroise, C., McLachlan, G.J.: Selection bias in gene extraction on the basis of microarray gene-expression data. Proceedings of the National Academy of Sciences **99**(10), 6562–6566 (2002) https://doi.org/10.1073/pnas.102102699

[23] Bergstra, J., Bengio, Y.: Random

search for hyper-parameter optimization. Journal of Machine Learning Research **13**, 281–305 (2012) https://doi.org/10.5555/2188385.2188395

[24] Bergstra, J.S., Bardenet, R., Bengio, Y., Kégl, B.: Algorithms for hyper-parameter optimization. In: Advances in Neural Information Processing Systems 24 (2011). https://doi.org/10.5555/2986459.2986743

[25] Jones, D.R., Schonlau, M., Welch, W.J.: Efficient global optimization of expensive black-box functions. Journal of Global Optimization **13**(4), 455–492 (1998) https://doi.org/10.1023/A:1008306431147

[26] Snoek, J., Larochelle, H., Adams, R.P.: Practical bayesian optimization of machine learning algorithms. In: Advances in Neural Information Processing Systems 25 (2012). https://doi.org/10.5555/2999325.2999464

[27] Hutter, F., Hoos, H.H., Leyton-Brown, K.: Sequential model-based optimization for general algorithm configuration. In: Coello, C.A.C. (ed.) Learning and Intelligent Optimization, pp. 507–523. Springer, Berlin, Heidelberg (2011). https://doi.org/10.1007/978-3-642-25566-3_40

[28] Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., Talwalkar, A.: Hyperband: A novel bandit-based approach to hyperparameter optimization. Journal of Machine Learning Research **18**(185), 1–52 (2018) https://doi.org/10.5555/3291125.3291310

[29] Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M.: Optuna: A next-generation hyperparameter optimization framework. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 2623–2631 (2019). https://doi.org/10.1145/3292500.3330701

[30] Feurer, M., Hutter, F.: Hyperparameter optimization. In: Hutter, F., Kotthoff, L., Vanschoren, J. (eds.) Automated Machine Learning, pp. 3–33. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-05318-5_1

[31] Hooke, R., Jeeves, T.A.: "Direct Search" solution of numerical and statistical problems. Journal of the ACM **8**(2), 212–229 (1961) https://doi.org/10.1145/321062.321069

[32] Nelder, J.A., Mead, R.: A simplex method for function minimization. The Computer Journal **7**(4), 308–313 (1965) https://doi.org/10.1093/comjnl/7.4.308

[33] Torczon, V.: On the convergence of pattern search algorithms. SIAM Journal on Optimization **7**(1), 1–25 (1997) https://doi.org/10.1137/S1052623493250780

[34] Lewis, R.M., Torczon, V.: Pattern search algorithms for bound constrained minimization. SIAM Journal on Optimization **9**(4), 1082–1099 (1999) https://doi.org/10.1137/S1052623496300507

[35] Audet, C., Dennis, J.E. Jr.: Mesh adaptive direct search algorithms for constrained optimization. SIAM Journal on Optimization **17**(1), 188–217 (2006) https://doi.org/10.1137/040603371

[36] Kolda, T.G., Lewis, R.M., Torczon, V.: Optimization by direct search: New perspectives on some classical and modern methods. SIAM Review **45**(3), 385–482 (2003) https://doi.org/10.1137/S003614450242889

[37] Conn, A.R., Scheinberg, K., Vicente, L.N.: Introduction to Derivative-Free Optimization. Society for Industrial and Applied Mathematics, Philadelphia, PA (2009). https://doi.org/10.1137/1.9780898718768

[38] Audet, C., Hare, W.: Derivative-Free and Blackbox Optimization. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-68913-5

[39] Kelley, C.T.: Implicit Filtering. Society for Industrial and Applied Mathematics, Philadelphia, PA (2011). https://doi.org/10.1137/1.9781611971903

[40] Byrd, R.H., Lu, P., Nocedal, J., Zhu, C.:

A limited memory algorithm for bound constrained optimization. SIAM Journal on Scientific Computing **16**(5), 1190–1208 (1995) https://doi.org/10.1137/0916069

[41] Zhu, C., Byrd, R.H., Lu, P., Nocedal, J.: Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization. ACM Transactions on Mathematical Software **23**(4), 550–560 (1997) https://doi.org/10.1145/279232.279236

[42] Friedman, J., Hastie, T., Tibshirani, R.: Regularization paths for generalized linear models via coordinate descent. Journal of Statistical Software **33**(1), 1–22 (2010) https://doi.org/10.18637/jss.v033.i01